

Quicksort

Lec13

Meta-note: Objectives

- You might well have seen quicksort before - and could easily look it up anyway if just want “the algorithm”.
- So, the slides are organised to
 - give lots of motivations
 - and to see a process under which “you might have invented it”

Motivations

- In merge sort the `divide` is simple, and the `merge` (relatively) complicated
- Can we swap this and make the `merge` (very) simple?
 - Answer: make the `divide` more complicated so that the merge becomes `concatenate`
- Rough Analogy: sort a pack of cards by
 1. divide into `red` and `black` cards
 2. divide by suit (red into hearts and diamonds,...)
 3. divide by value ...

Then just stack the packs on top of each other

When is 'merge' simple?

- When the lists A and B are sorted and known to be in disjoint ordered ranges
 - all of elements of A are smaller than all those of B
- If A and B are stored as consecutive sub-arrays, then merge needs no work at all:
 - Just "forget the boundary" to get one sorted array "AB"

A					B			
2	7	9	10	10	12	13	15	17

- becomes

AB								
2	7	9	10	10	12	13	15	17

Quicksort: 2-way division

- It is a “divide and conquer” algorithm
 - The divide is designed so that the merge will be simply “forget the boundary”
- Hence, if divide into two subarrays, L and R, we require that no element in L is larger than any element in R
 - After sorting each of L and R, by recursing, then the merge is trivial

Quicksort: 2-way division (2)

Initial pseudo-code

```
qsort(int[] A) {  
    int b_LR = partition(A)  
    qsort(L) // L indices are < b_LR  
    qsort(R) // R indices are >= b_LR  
}
```

Where partition(A) takes the subarray and splits it into two subarrays L and R

- Moves elements so anything in L is no larger than anything in R
- Returns the “boundary” b_LR

Quicksort: partition example

Where partition(A) takes the subarray and splits it into two subarrays L and R

- finds boundary b_LR
- Moves elements so anything left is no larger than anything right

• E.g.

A							
2	17	13	10	12	10	13	9

- Might become

L				R			
2	10	9	10	17	13	13	12

- We want that L and R are roughly the same size
 - will see later this helps to give a good Big-Oh for the sort

Quicksort: partition checking

Where $\text{partition}(A)$ takes the subarray and splits it into two subarrays L and R

- finds boundary b_{LR}
- Moves elements so anything left is no larger than anything right
- How do we check we have a valid partition?
- Inefficient: compare all pairs of from (L,R)
 - $O(n^2)$
- Efficient: find max of L, and min of R, and check $\max(L) \leq \min(R)$
 - $O(n)$ – which is good as matches the cost of merge in merge sort
 - Note: this says nothing about where in L $\max(L)$ occurs – it could be anywhere in L
- Suggests doing the partition based on some number
 - Such a number will be called the “**pivot**”

Quicksort: equal-partition?

- Example

A							
2	17	13	10	12	10	13	9

- Will become sorted to become

A							
2	9	10	10	12	13	13	17

- We would like that the last merge step was from

L				R			
2	9	10	10	12	13	13	17

- Note that the median of the numbers in A is $(10+12)/2=11$
- As a consequence:
 - elements in L are all < 11
 - elements in R are all > 11
- So maybe the “divide” could use this as a criterion for moving the elements?

Quicksort: equal-partition?

- If we start from the array

A							
2	17	13	10	12	10	13	9

- And require that split into L and R with pivot=11
 - elements in L are all $<$ pivot
 - elements in R are all $\geq$ pivot
- Then this is easy to do using auxiliary workspaces L and R

foreach k in A

if $A[k] < \text{pivot}$ L.add($A[k]$)

else R.add($A[k]$)

then copy the L and R back into original array A

- Giving

L				R			
2	10	10	9	17	13	12	13

- But we want to avoid using such extra workspace

“In-place” vs “extra workspace”?

- For sorting algorithms (and algorithms in general) an important issue can be how much extra working space they need besides the space to store the input
- **“In-place”** means they only a “little” extra space above what is used to store data elements.
 - “A little extra space” usually means $O(1)$
 - The input array is also used for output, and only need a few temporary variables
 - Exercise: check that bubble-sort is “in-place”
 - Previous “merge” of mergesort used an extra $O(n)$ array as “workspace”
 - (can be made in-place, but messy and so we ignore this option)

Problems to address:

- Can we do the partition (by the “pivot”) in-place?
- What is the problem with using pivot=median?
 - How can we (partially) fix it?

Quicksort: in-place partition?

- If we start from the array

A							
2	17	13	10	12	10	13	9

- And require that split into L and R with **pivot=11**
 - elements in L are all $<$ pivot
 - elements in R are all $\geq$ pivot
- Then we can work from both ends inwards looking for swaps whenever the entries are “in the wrong order”
 - Maintain indices kL and kR
 - kL starts at the left end and moves right kL++
 - kR starts at the right end and moves left kR--
- Start:
 - A[kL]=2 and $2 <$ pivot, and is on the left, so it is okay, Can leave A[0]=2 alone. So just do kL++, getting kL=1
 - A[kR]=9 and $9 <$ pivot. At this point it could be okay to leave it – if all elements were “ $<$ pivot”, but maybe it will need moving. So just leave kR unchanged for the moment.

Quicksort: first swap

0	1	2	3	4	5	6	7
2	17	13	10	12	10	13	9

- Next:
 - $kL=1$ $A[kL]=17$ and $\text{pivot} \leq 17$, and so should be in R
 - We still have $kR=7$, $A[kR]=9$ and $9 < \text{pivot}$ and should be in L.
- At this point we have a pair (17,9) that are the “wrong way around” – the 17 should be on the right, and the 9 in the left.
- The fix is to simply swap the 17 and 9 entries, giving

0	1	2	3	4	5	6	7
2	9 17	13	10	12	10	13	9 17

- Now $kR=7$ and $\text{pivot} \leq A[kR]$ is “in the right place” and so can do $kR--$, moving it left to $kR=6$
- Now $kL=1$ and $A[kL] < \text{pivot}$ is “in the right place” and so can do $kL++$, moving it right to $kL=2$

Quicksort: second swap

0	1	2	3	4	5	6	7
2	9	13	10	12	10	13	17

- Now we have
 - $kL=2$, $A[kL]=13$ and $\text{pivot} \leq 13$, and so should be in R. Hence pause.
 - $kR=6$, $A[kR]=13$ and $\text{pivot} \leq 13$ pivot and should be in R, so can do $kR--$
 - $kR=5$, $A[kR]=10$ and $10 < \text{pivot}$, and so should be in L.
- At this point we have a pair (13,10) that are the “wrong way around” – the 13 should be on the right, and the 10 in the left.
- The fix is to simply swap the entries at indices (2,5) giving

0	1	2	3	4	5	6	7
2	9	10	13	12	10	13	17

- Now $kR=5$ and $\text{pivot} \leq A[kR]$ is “in the right place” and so can do $kR--$, moving left to $kR=4$
- Now $kL=2$ and $A[kL] < \text{pivot}$ is “in the right place” and so can do $kL++$, moving right to $kL=3$

Quicksort: third swap?

0	1	2	3	4	5	6	7
2	9	13 10	10	12	10 13	13	17

- Now we have
 - $kL=3$, $A[kL]=10$ and $10 < \text{pivot}$, and so should be in L. Hence can increment $kL++$ to get $kL=4$
 - $kR=4$, $A[kR]=12$ and $\text{pivot} \leq 12$ pivot and should be in R,
 - So do not need to swap.
 - Also, the indices have now met as $kL=kR=4$
- We did not need to do a third swap, and there is no need for further swaps. We have finished with array state

0	1	2	3	4	5	6	7
2	9	10	10	12	13	13	17

- The finishing indices $kL=kR=4$ define
 - the start of the "R" partition with all element $\geq \text{pivot}$
 - the end of the "L" partition with all elements $< \text{pivot}$
- Note: it is accidental that L and R are sorted – not true in general

We can partition in-place given pivot

- Algorithm works by moving
 - Leftmost index rightwards
 - Rightmost index leftwards
- whilst looking for swaps.
- The movements are independent – they are not forced lockstep
- We only had to
 - Maintain 2 indices k_L and k_R
 - Do swaps, needing one temp variable
- Hence, we only needed $O(1)$ extra space
- It also traverse the array once – so is $O(n)$ time
- Hence, given a choice of pivot, we can partition using $O(n)$ time and $O(1)$ space
- Note: we could do either of
 - “ $\geq$ pivot” for the right, “ $<$ pivot” for the left
 - “ $>$ pivot” for the right, “ $\leq$ pivot” for the left

Choosing the median as pivot

- Best choice: If pick $\text{pivot} = \text{median}$ then the partitions will have equal size, and takes work $O(n)$ to do the partition
- Mergesort was also characterised by having equal size partitions – and also $O(n)$ work at each level
 - In this case the algorithm efficiency analysis is the same as for mergesort
- The trace of the running will be a binary tree of height $\log_2(n)$ and with work $O(n)$ at each depth
 - Hence overall $O(n \log n)$

Alternatively:

- We will get the same recurrence relation
 - $T(n) = 2 T(n/2) + a n + b$
 - Except the linear term “ $a n$ ” is now for the partition not the merge
 - So, Master Theorem immediately gives $\Theta(n \log n)$
- But what is the catch !?

Choosing the pivot: Random

- Catch: There is no efficient way to find the median of an array !
- We could find the median by sorting the array
 - But then we would have to sort – so get no gain
- Fix? we can try to estimate the median, or something that is “good enough”
- Simplest is just to take a random element p of the array.
 - I.e. pick $p = \text{rand}(0, \dots, A.\text{size}-1)$ and take
 - Pivot = $A[p]$
- Then $1/3$ of the attempts, the division will still that the smallest partition is at least $1/3$ of the current size.
 - Then the height of the “trace tree” will still be logarithmic – as $\log_3$ and $\log_2$ only differ by a constant

Choosing the pivot: Partial Random

- A better estimate of the median would be to take 3 different random elements of the array; then take the median of those.
- Even better would be to take the median of 5 randomly selected elements
- Etc.
- There can be many choices, and different techniques to select a good value for the pivot

Choosing the pivot: Worst case

- In the worst case, the selected pivot could be the minimum or maximum of the array, and then the best we could do is to have the pivot in one partition and all the other elements in the other partition
- In this case the “trace tree” will have height $O(n)$
- Each level still takes $O(n)$ work
- Hence quicksort degenerates to $O(n^2)$
- The pivot is the min/max and the partition just moves the pivot to the correct end of the array
 - Hence each step basically ends like one pass of the inner loop of bubble or selection sort

Quicksort: runtimes

- Best case: $O(n \log n)$
 - Pivot is guaranteed to be (close enough to) the median
- Average case: probabilistic selection of a pivot leads to a “good partition” sufficiently often that still have $O(n \log n)$
 - You should know this, but do not need to be able to prove it
- Worst case: pivot is always bad – partition makes minimal progress
 - Degenerates to $O(n^2)$

Summary: Motivations for quicksort

- What advantages might quicksort ever have over mergesort?
 - Because it can be done “in-place”
 - Uses a small amount of workspace
 - Because the “merge” step is now a lot easier!!
 - The “split” is more complicated, and the merge “much” easier – but turns out that the quick-sort split is easier to do in-place than the merge within merge-sort
 - Might be a good choice if workspace is very limited

Exercises (offline)

- Investigate whether quicksort is stable
 - Do equal elements ever get swapped
- Investigate whether quicksort is adaptive
 - E.g. if the input is already sorted then does the algorithm complexity improve?

Minimum Expectations

- For both merge- and quicksort:
 - know the algorithm and how it works on examples
 - know and be able to justify/prove their big-Oh behaviours
- Exercise (offline) : do many examples of the in-place partition until you are clear how it works